《Ai Agent》第3-1节，Ai Agent 业务流程、系统架构、库表设计说明

来自： 码农会锁

小傅哥 2025年05月24日 18:30

本章重点： ★★★★☆
课程视频： https://t.zsxq.com/DcL2p
代码分支：master
工程代码： https://gitcode.net/KnowledgePlanet/ai-agent-station

版权说明： ©本项目与星球签约合作，受《中华人民共和国著作权法实施条例》版权法保护，禁止任何理由和任何方式公开(public)源码、资料、视频等小傅哥发布的星球内容到Github、Gitee等各类平台，违反可追究进一步的法律责任。

作者：小傅哥
博客：https://bugstack.cn

> 沉淀、分享、成长，让自己和他人都能有所收获！😄

一、本章诉求
二、Agent 介绍
三、Agent 设计
四、Agent 代码
五、Agent 流程
1. 功能流程
2. 代码流程
六、Agent 库表
七、Agent 工程
八、读者作业

一、本章诉求

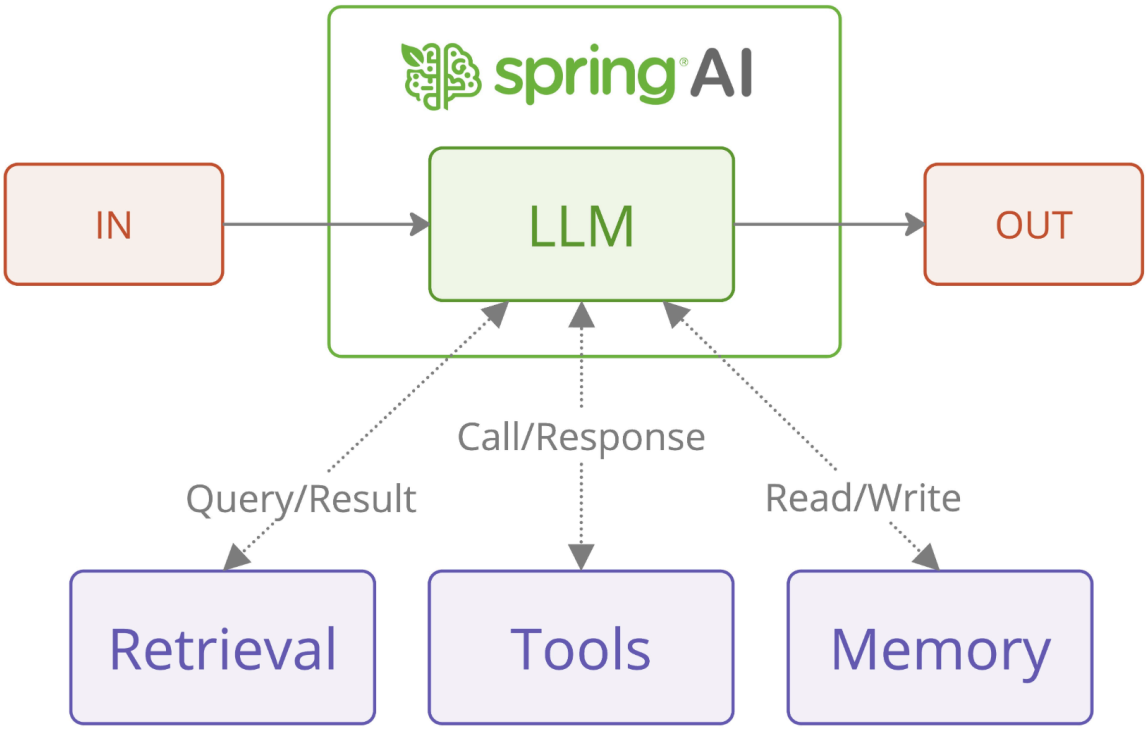
了解 Ai Agent 构建元素，以 Spring AI 硬编码创建 Agent 过程，分析各个模块用途，并以此设计拆分元素和对应的库表设计。
经过本节我们可以了解到，怎么把这些硬编码的过程，以配置数据库表的方式，动态化构建。有了动态化的构建，也就可以按需配置出各种 Agent 服务来满足我们的业务诉求。尤其是现在 MCP 如火如荼的发展，有一套自动化的 Agent 是非常重要的。

> 第3-0节，介绍和演示中，涉及了本节的部分内容。本节主要站在开发视角，来讲解如何架构和开发系统。

二、Agent 介绍

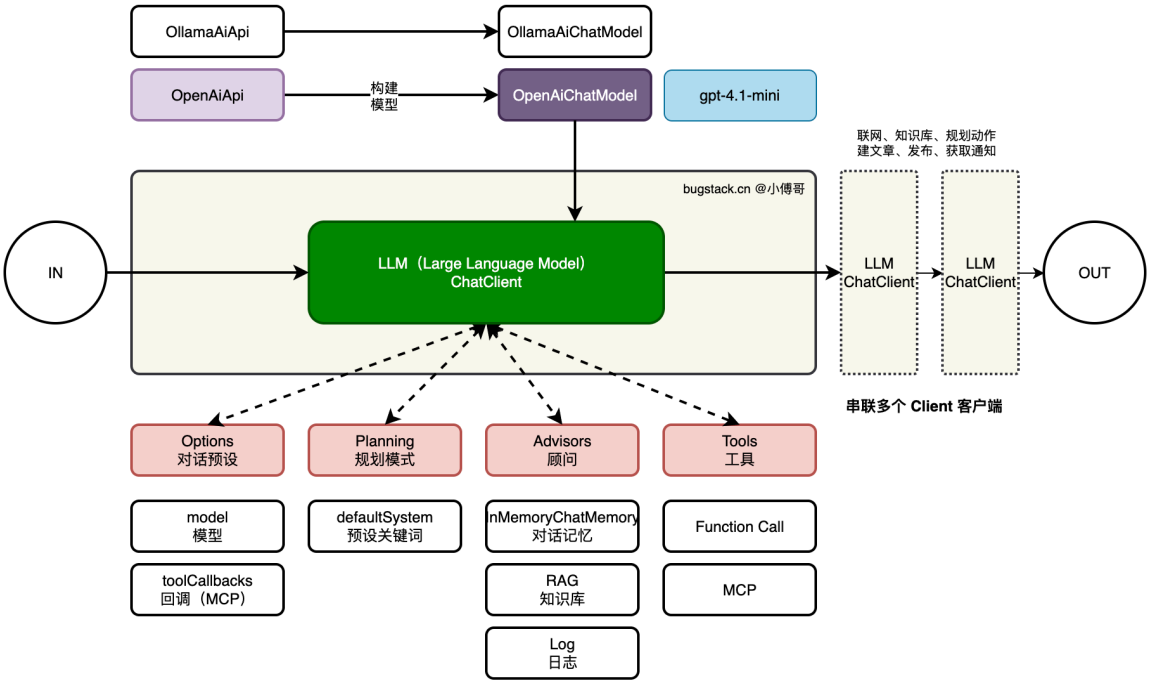
AI 智能体是使用 AI 来实现目标并代表用户完成任务的软件系统。其表现出了推理、规划和记忆能力，并且具有一定的自主性，能够自主学习、适应和做出决定。

这些功能在很大程度上得益于生成式 AI 和 AI 基础模型的多模态功能。AI 智能体可以同时处理文本、语音、视频、音频、代码等多模态信息；可以进行对话、推理、学习和决策。它们可以随着时间的推移不断学习，并简化事务和业务流程。智能体可以与其他智能体协作，来协调和执行更复杂的工作流。



> Spring AI 框架，支持大语言模型构建 AI Agent 实现。AI Agent是整合多种技术手段的智能实体 ，其实现依赖于 Tools、MCP、Memory、RAG（Retrieval 增强检索生成） 等技术组件，但不是非得依赖全部组件才叫 AI Agent。

三、Agent 设计



如图，以 ChatClient 为中心，设置；Ai模型、对话预热、规划模式、顾问、工具，完成 LLM ChatClient 构建。这些注入的元素属性，驱动着 Agent 功能的实现。

会话时，从输入端到输出端，可以顺序执行多个 ChatClient。通过这样的方式，让不同配置的 ChatClient 发挥自身作用，驱动最终目标的完成。

对话预设；这个节点在 ChatModel 中添加，可以设置对话的 AI 模型，也可以设置 MCP。ChatModel 可以支持流式对话。还可以把 ChatModel 注入到 ChatClient 中使用。

规划模式；为预设的关键词，让一个步骤流程知道以哪些关键信息进行运行。

顾问角色；在 Spring AI 中提供了一种更加优雅的访问外部元素信息的方式，顾问访问。通过顾问功能，增强对话。包括；上下文记忆、知识库、日志等。

工具加载；这里我们主要使用 MCP，也是目前市面最为流行的方式。本项目中用到的 MCP 就包括 AI 课程中第2阶段实现的 MCP 服务。

四、Agent 代码

```
@Slf4j
@RunWith(SpringRunner.class)
@SpringBootTest
public class AiAgentTest {

    @Before
    public void init_client() {
        OpenAiChatModel chatModel = OpenAiChatModel.builder()
                .openAiApi(OpenAiApi.builder()
                        .baseUrl("https://apis.itedus.cn")
                        .apiKey("sk-lIqVNiHon00O6veJ15Cc57DaF5Dd401f93B3A107B4B3677e")
                        .completionsPath("v1/chat/completions")
                        .embeddingsPath("v1/embeddings")
                        .build())
                .defaultOptions(OpenAiChatOptions.builder()
                        .model("gpt-4.1")
                        .toolCallbacks(
                                FunctionToolCallback.builder("test",
                                                (Function<TestFunctionInput, String>) testFunction
                                                    log.info("函数请求:{}", testFunctionInput.getIn
                                                    return "王大瓜今天入职啦!";
                                                })
                                        .description("王大瓜")
                                        .inputType(TestFunctionInput.class)
                                        .build())
                        .toolCallbacks(new SyncMcpToolCallbackProvider(sseMcpClient01(), sseMcpCli
                        .build())
                .build();

        ChatClient chatClient = ChatClient.builder(chatModel)
                .defaultSystem("""
                          你是一个 AI Agent 智能体，可以根据用户输入信息生成文章，并发送到 CSDN 平台以及完成微

                          你擅长使用Planning模式，帮助用户生成质量更高的文章。

                          你的规划应该包括以下几个方面：
                          1. 分析用户输入的内容，生成技术文章。
                          2. 提取，文章标题（需要含带技术点）、文章内容、文章标签（多个用英文逗号隔开）、文章简述
                          3. 获取发送到 CSDN 文章的 URL 地址。
                          4. 微信公众号消息通知，平台：CSDN、主题：为文章标题、描述：为文章简述、跳转地址：从发
                        """)
                .defaultToolCallbacks(new SyncMcpToolCallbackProvider(sseMcpClient01(), sseMcpClie
                .defaultAdvisors(
                        new PromptChatMemoryAdvisor(new InMemoryChatMemory()),
                        new RagAnswerAdvisor(vectorStore, SearchRequest.builder()
                                .topK(5)
                                .filterExpression("knowledge == '知识库名称'")
                                .build()),
                        new SimpleLoggerAdvisor()
                )
                .defaultOptions(OpenAiChatOptions.builder()
                        .model("gpt-4.1")
                        .toolCallbacks(
                                FunctionToolCallback.builder("test",
                                                (Function<TestFunctionInput, String>) testFunction
                                                    log.info("函数请求:{}", testFunctionInput.getIn
                                                    return "王大瓜今天入职啦!";
                                                })
                                        .description("王大瓜")
                                        .inputType(TestFunctionInput.class)
                                        .build())
```

```
                        .build())
                .build();
    }

    public McpSyncClient sseMcpClient02() {

        HttpClientSseClientTransport sseClientTransport = HttpClientSseClientTransport.builder("ht

        McpSyncClient mcpSyncClient = McpClient.sync(sseClientTransport).requestTimeout(Duration.o

        var init = mcpSyncClient.initialize();
        System.out.println("SSE MCP Initialized: " + init);

        return mcpSyncClient;
    }

    public McpSyncClient stdioMcpClient() {

        // based on
        // https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem
        var stdioParams = ServerParameters.builder("npx")
                .args("-y", "@modelcontextprotocol/server-filesystem", "/Users/fuzhengwei/Desktop"
                .build();

        var mcpClient = McpClient.sync(new StdioClientTransport(stdioParams))
                .requestTimeout(Duration.ofSeconds(10)).build();

        var init = mcpClient.initialize();

        System.out.println("Stdio MCP Initialized: " + init);

        return mcpClient;

    }

    @Test
    public void test_client02() {
        String userInput = "生成一篇文章";
        System.out.println("\n>>> QUESTION: " + userInput);
        System.out.println("\n>>> ASSISTANT: " + chatClient
                .prompt(userInput)
                .system(s -> s.param("current_date", LocalDate.now().toString()))
                .advisors(a -> a
                        .param(CHAT_MEMORY_CONVERSATION_ID_KEY, "chatId-101")
                        .param(CHAT_MEMORY_RETRIEVE_SIZE_KEY, 100))
                .call().content());
    }

    @Test
    public void test_clien03() {
        String userInput = "有哪些工具可以使用";
        System.out.println("\n>>> QUESTION: " + userInput);
        System.out.println("\n>>> ASSISTANT: " + chatClient
                .prompt(userInput)
                .system(s -> s.param("current_date", LocalDate.now().toString()))
                .advisors(a -> a
                        .param(CHAT_MEMORY_CONVERSATION_ID_KEY, "chatId-101")
                        .param(CHAT_MEMORY_RETRIEVE_SIZE_KEY, 100))
                .call().content());
    }

}
```

首先，这部分代码，其实就是 Ai Agent 自动化构建的雏形，整个过程包括了模型的创建、工具（MCP）的加载、预设的话术、顾问角色处理上下文记忆和知识库，再到构建客户端。以及完成客户端的验证。

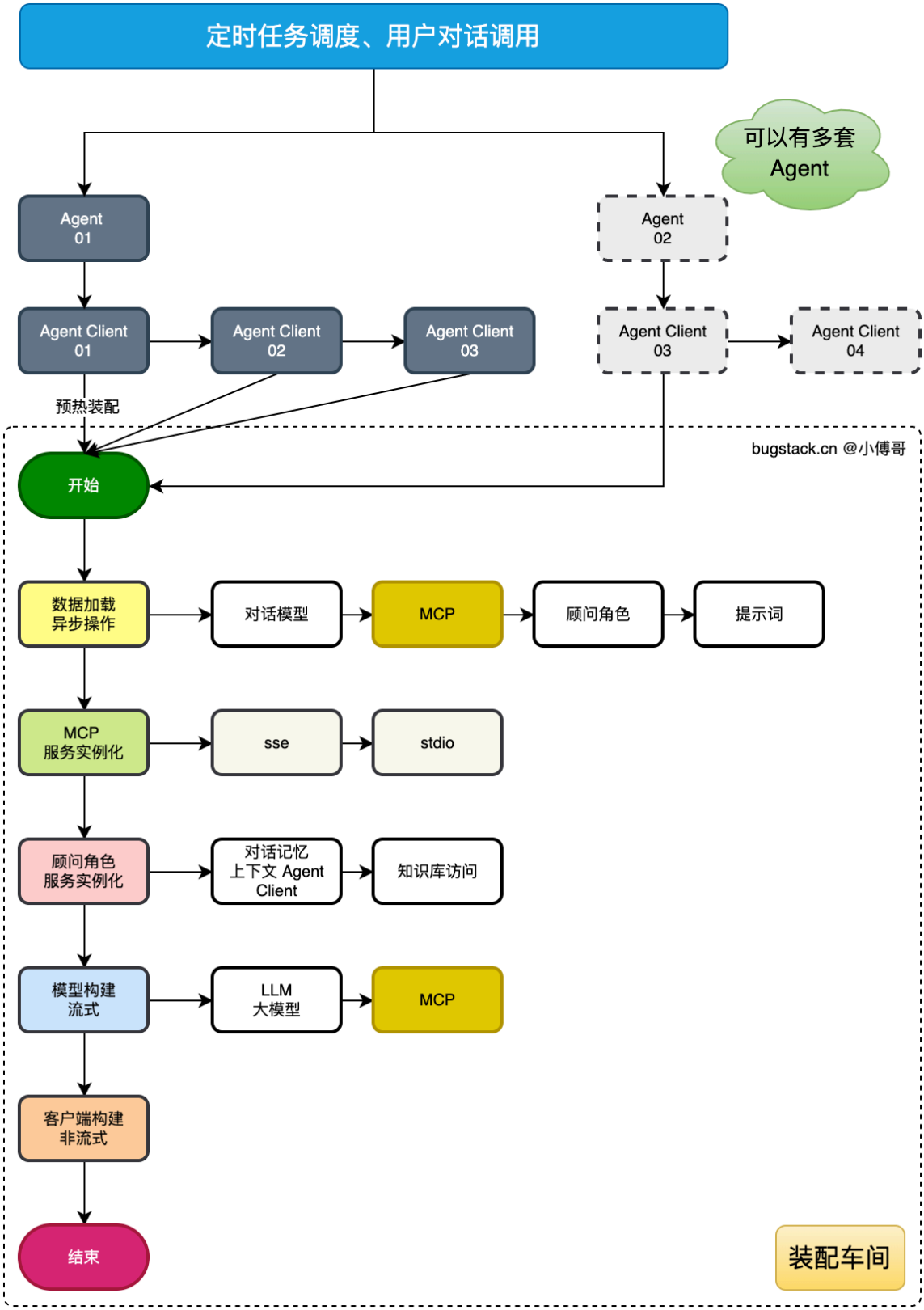
那么，我们这套代码就是要把所有的这些硬编码进行解耦，通过数据库配置的方式 + 程序员自动化的处理，完成各个 Bean 对象的构建和使用。这样我们就可以随意的配置出任何一套，需要走知识库、MCP和对应预设话术，完成我们指定业务类型的 Agent 服务。

并且，这样的 Agent 服务，可以串联多个 Agent，以及让自动化的 job 驱动和人工对话的方式完成日常的功能。

想想，有了这样一套东西，你就可以让 Ai Agent 帮你完成很多的日常的工作。如；系统监控巡检、异常问题根因分析、日常活动自动运营、需求代码前置分析、自动客户端对话等等。

五、Agent 流程

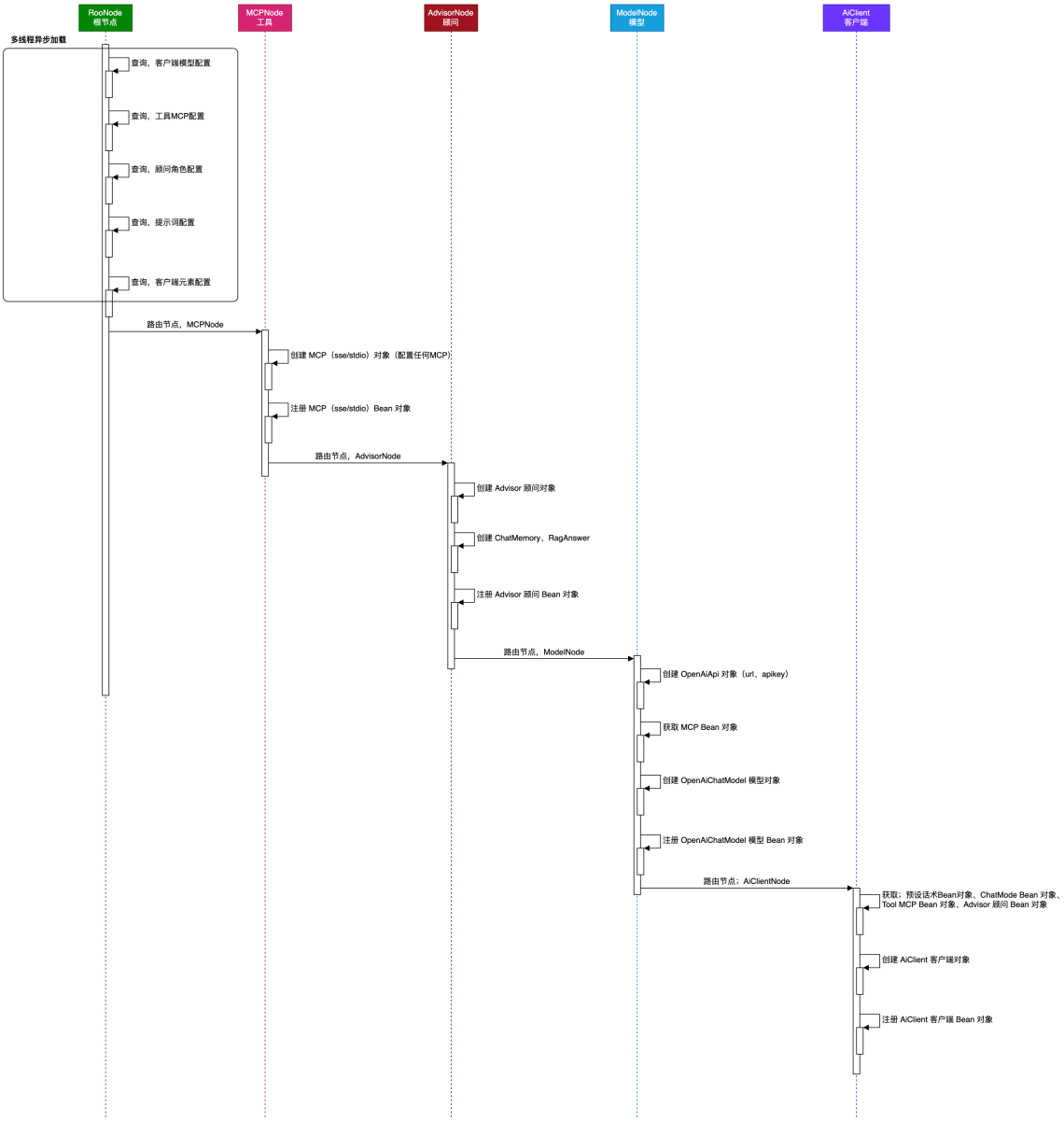
1. 功能流程



如图，我们把整个代码的构建过程进行拆分。上半部分是 Agent 功能的编排和触达，下半部分是每个 Agent Client 的预热装配过程。这个装配是基于数据库表信息的配置，之后在应用程序启动和管理后台人工触达的方式，完成 Agetn Client 的构建。

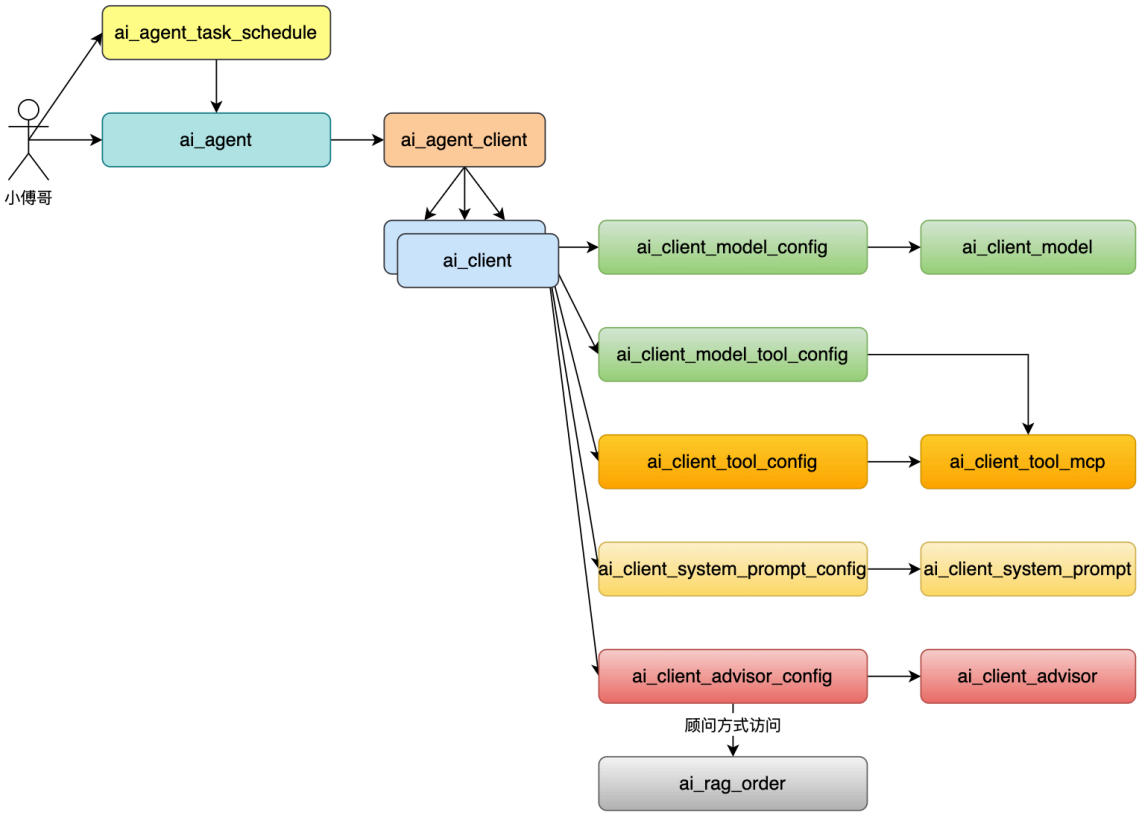
构建包括：数据异步加载、MCP 服务实例化、顾问角色、模型构建、客户端构建。整个构建过程引入了星球里扳手工程，使用扳手里的设计模式框架的规则模型进行编排这些流程。这样以后再有其他流程，我们也可以非常容易的扩展进来。

2. 代码流程



如图，这个是整个 Agent 动态化创建的核心流程，最早侧先多线程方式加载数据，之后依次创建；MCP工具、顾问角色、对话模型，最后创建 Client 客户端。 – 这个流程采用的是星球的扳手工程，提供的设计模式框架处理的。https://t.zsxq.com/veRkQ

六、Agent 库表



如图，设计了14张库表，来承接自动化 Ai Agent 的构建操作。

ai_agent_task_schedule，智能体任务调度配置表

ai_agent，AI智能体配置表

ai_agent_client，智能体-客户端关联表

ai_client，AI客户端配置表

模型配置组；ai_client_model、ai_client_model_config、ai_client_model_tool_config

工具配置组；ai_client_tool_config、ai_client_tool_mcp

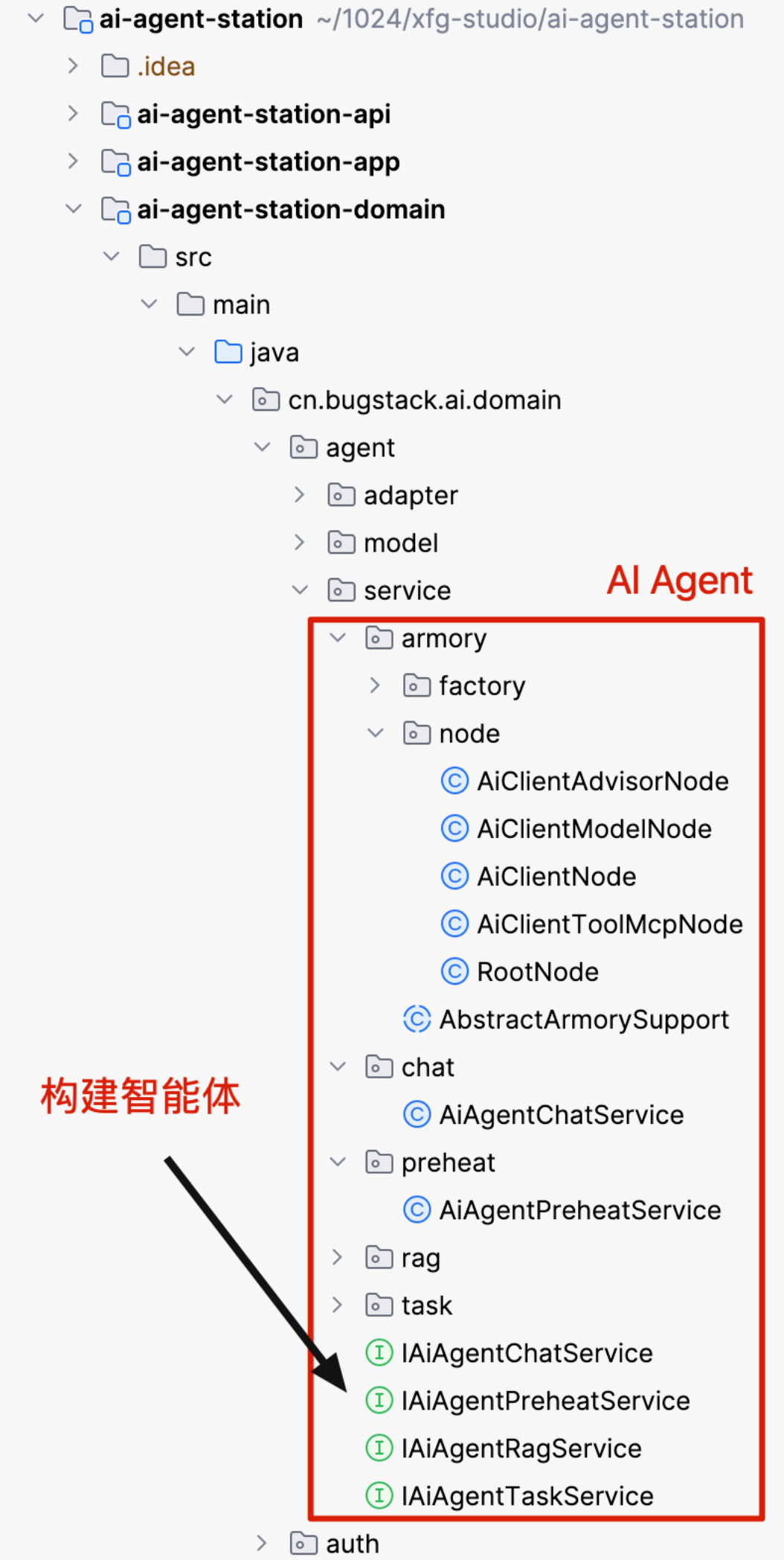
顾问配置组；ai_client_advisor、ai_client_advisor_config

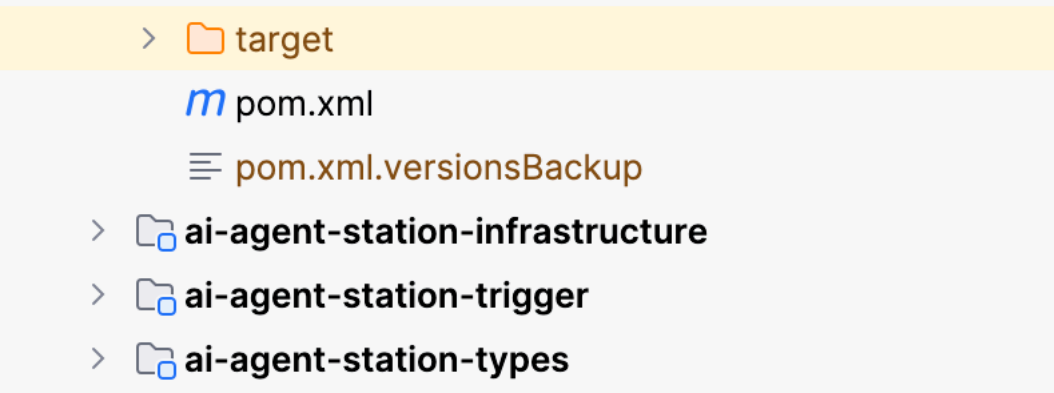
提示词配置；ai_client_system_prompt、ai_client_system_prompt_config

知识库配置；ai_rag_order

> 建表语句会在后续课程中提供，学习到本节，可以先尝试自己创建。

七、Agent 工程


ai-agent-station ~/1024/xfg-studio/ai-agent-station
.idea
ai-agent-station-api
ai-agent-station-app
ai-agent-station-domain
src
main
java
cn.bugstack.ai.domain
agent
adapter
model
service
AI Agent
armory
factory
node
AiClientAdvisorNode
AiClientModelNode
AiClientNode
AiClientToolMcpNode
RootNode
AbstractArmorySupport
chat
AiAgentChatService
preheat
AiAgentPreheatService
rag
task
IAiAgentChatService
IAiAgentPreheatService
IAiAgentRagService
IAiAgentTaskService
构建智能体
auth



如图，为整个系统的工程结构，分为 api、app、domain、infrastructure、trigger、types，六边形架构。（现在各个互联网都在落地 DDD，因为 DDD 比最早出来的几年，已经有了非常明确的规范）相关资料；https://bugstack.cn/md/road-map/ddd-guide-01.html

Domain 核心领域层，处理 Agent 的预热、对话、知识库、任务的操作。后续 Agent 相关都维护到这个领域包下。

Trigger 触发器层，负责对外提供接口，让外部来调用。当有一些纯 crud 操作的流程时，这个架构下，会在 trigger 层直接调用基础设施层提供数据，而不需要在经过 domain 领域层，重复封装对象。

八、读者作业

简单作业：理解 Agent，以及学习构建 Agent 的设计，代码，流程，库表设计和工程结构。

复杂作业：尝试自己创建初始工程，按照流程设计来实现。

知识星球